Entrega #4

Giuliano Pascarelli

3 de noviembre de 2026

UNIVERSIDAD SIGLO 21

La educación evoluciona

M4 - Giuliano Pascarelli
Licenciatura en Informática.
Materia: SEMINARIO DE PRÁCTICA DE INFORMÁTICA (INF275-11987)
Período académico: 2026, subperíodo 2B
Profesor: MARCOS DARIO ARANDA

1. Índice

Tabla de contenidos

1 Índice **2**

2 Consigna **3**
- 2.1 Enunciado . . . 3
- 2.2 Consignas . . . 3

3 Desarrollo **4**
- 3.1 Introducción . . . 4
- 3.2 Entregas anteriores (corregidas) . . . 4
- 3.3 Explicación y Justificación del Patrón de Diseño . . . 4
- 3.4 Persistencia y Base de Datos (JDBC) . . . 4
- 3.5 Justificación de Decisiones Técnicas . . . 5
  - 3.5.1 Excepciones . . . 5
  - 3.5.2 Uso de `ArrayList` . . . 5
  - 3.5.3 Manejo de Archivos . . . 5

2. Consigna

2.1. Enunciado

Considera que te encuentras trabajando en la organización cuyo problema definiste resolver con un proyecto informático, y te solicitan liderar el equipo de desarrollo que va a llevar adelante el mismo.
El proyecto informático que definas debe ser distinto al considerado en la lectura y el entregable final debe cumplir con los siguientes objetivos:

- Realizar el análisis del modelo de negocios para definir y justificar el proyecto.
- Aplicar el proceso unificado de desarrollo (PUD) para garantizar la calidad, escalabilidad y eficiencia en el ciclo de desarrollo.
- Utilizar una base de datos MySQL para la persistencia de los datos.
- Emplear Java como lenguaje de programación para el desarrollo del sistema.

A los fines del trabajo, para la implementación de la base de datos y el desarrollo con Java, puedes presentar un prototipo, que es "es la creación de un modelo operacional que incluya solo algunas características del sistema final" (Kendall & Kendall, 2011).

El concepto de operacional es clave, ya que no se trata de un simple modelo, sino que permite desarrollar módulos que se van integrando en la versión final del sistema.

Esta cuarta actividad te permitirá avanzar con el proyecto integrador que debe utilizar un patrón de diseño coherente con las definiciones establecidas en la arquitectura definida.

Para el desarrollo del prototipo en Java, deberás crear un paquete cuyas clases permitan realizar la persistencia en una base de datos MySQL. Esta actividad incluye establecer conexiones, realizar consultas, actualizar registros y presentar resultados en la interfaz. En todos los casos que se requiera, recuerda aplicar correctamente el manejo de excepciones.

El desarrollo debe incluir la aplicación de clases abstractas o interfaces. También debe contemplar la utilización de arreglos y emplear la clase ArrayList, ambas estructuras de forma complementaria.

Según las características del proyecto, puedes utilizar archivos. Aunque esta actividad es opcional, te recomiendo practicar su aplicación, previo a presentarte en el EFIP I.

2.2. Consignas

Los entregables a presentar son los siguientes:

1. Presentación del desarrollo en Java.
2. Entrega del proyecto integrador, completo.
3. Presentación del proyecto en un video con una duración de aproximadamente 3 minutos.

El proyecto o prototipo desarrollado debe contemplar necesariamente las siguientes características:

1. Corrección de las observaciones realizadas en la entrega de las actividades anteriores.
2. Selección del patrón de diseño y justificación.
3. Persistencia y consulta de datos en una base de datos MySQL. Esta actividad incluye establecer conexiones, realizar consultas, actualizar registros y presentar resultados en la interfaz.
4. Correcta aplicación de excepciones para la interacción con la base de datos MySQL.
5. Inclusión pertinente de clases abstractas o interfaces.
6. Utilización complementaria de arreglos y de la clase ArrayList.

En esta instancia, te sugiero revisar la rúbrica para asegurar el cumplimiento del criterio expuesto que considera la evaluación de tu participación activa en actividades propuestas: foros, encuentros sincrónicos, aportes en espacios comunes y comunicación.

Te recomiendo, además, apoyarte en las lecturas y bibliografía propuesta en las materias: Programación Orientada a Objetos, Taller de Algoritmos y Estructuras de Datos I.

3. Desarrollo

3.1. Introducción

Enlace al proyecto: https://github.com/lelox028/Siglo21-Sistema-Fidelizacion
El repositorio público contiene el código fuente Java del sistema y los scripts SQL utilizados para crear, poblar y consultar la base de datos. El video de demostración se encuentra en el siguiente enlace: https://www.youtube.com/watch?v=example.
El presente informe documenta las decisiones técnicas adoptadas para completar el prototipo del Sistema de Fidelización de Clientes. La implementación conserva el alcance definido en las entregas anteriores: registrar clientes, administrar beneficios, asociar beneficios con compras y consultar información histórica mediante una aplicación de escritorio desarrollada en Java Swing, con persistencia en MySQL.

3.2. Entregas anteriores (corregidas)

Las entregas correspondientes a AP1, AP2 y AP3 fueron aprobadas sin correcciones. Por este motivo, no se realizan modificaciones sobre los contenidos ya evaluados: se mantiene la propuesta del sistema de fidelización, el análisis y los requerimientos definidos en AP1, la arquitectura, el modelo de datos y las decisiones de diseño de AP2, y la implementación orientada a objetos documentada en AP3.
Los informes anteriores serán adjuntados nuevamente junto con esta entrega para facilitar su revisión y permitir la consulta de los fundamentos, modelos y decisiones que sirven de base para el desarrollo presentado en AP4.

3.3. Explicación y Justificación del Patrón de Diseño

El sistema utiliza el patrón arquitectónico MVC (*Model–View–Controller*), adaptado a la organización en paquetes del proyecto. Esta elección permite separar las responsabilidades y evitar que la interfaz gráfica, la lógica de negocio y el acceso a los datos queden concentrados en las mismas clases.

- **Modelo:** está representado por las clases del paquete `Modelo`, como `Cliente`, `Compra`, `Beneficio` y `BeneficioAplicado`. Estas clases encapsulan los datos y representan las entidades principales del dominio.
- **Vista:** está formada por las clases del paquete `Vista`, construidas con Java Swing, como `PantallaPrincipal`, `FormularioCliente`, `FormularioCompra`, `FormularioBeneficio` y `PantallaReportes`. Su responsabilidad es presentar la información, recibir las acciones del usuario y mostrar los resultados.
- **Controlador:** corresponde principalmente a las clases `GestorClientes`, `GestorCompras`, `GestorBeneficios`, `GestorAplicacionBeneficios` y `GestorReportes`, ubicadas en el paquete `Control`. Los Gestores coordinan las operaciones solicitadas por la Vista, validan reglas de negocio y delegan la persistencia a los DAOs.

El paquete `DAO` complementa esta separación al encapsular el acceso a MySQL. Aunque no constituye una de las tres partes nominales de MVC, permite que el Controlador no dependa directamente de sentencias SQL y mantiene aislados los detalles de infraestructura. De este modo, la Vista no necesita conocer cómo se almacenan los datos, el Modelo no contiene consultas SQL y los Gestores concentran la coordinación de los casos de uso.

3.4. Persistencia y Base de Datos (JDBC)

La aplicación se conecta con una base de datos relacional MySQL utilizando JDBC (*Java Database Connectivity*), sin frameworks de persistencia. JDBC proporciona las clases y métodos estándar para abrir una conexión, preparar sentencias SQL, enviar consultas o actualizaciones y recuperar los resultados devueltos por la base de datos.
La clase `ConexionDB`, ubicada en el paquete `DAO`, centraliza la creación y el cierre de la conexión. Obtiene la configuración desde variables de entorno, registra el controlador `com.mysql.cj.jdbc.Driver` y utiliza `DriverManager` para conectarse mediante la URL JDBC. La clase mantiene una única conexión activa y la vuelve a crear si no existe o se encuentra cerrada.

Las operaciones de persistencia se organizan mediante el patrón DAO (*Data Access Object*). Clases como `ClienteDAO`, `CompraDAO`, `BeneficioDAO` y `BeneficioAplicadoDAO` concentran las operaciones de inserción, búsqueda, actualización y listado de sus respectivas entidades. Para ejecutar SQL se utilizan `PreparedStatement`, parámetros y bloques *try-with-resources*, lo que permite separar el acceso a datos de las entidades y de la interfaz, además de asegurar el cierre de los recursos utilizados.
Cuando una operación JDBC produce un error, el DAO lo encapsula en `ErrorAccesoDatosException`. Luego, los Gestores pueden traducirlo a `ErrorOperacionException` para que la Vista reciba un error propio de la operación solicitada, sin depender de los detalles técnicos de JDBC. La base relacional mantiene además las relaciones entre clientes, compras y beneficios, junto con la integridad referencial y las propiedades ACID necesarias para las operaciones del sistema.

3.5. Justificación de Decisiones Técnicas

3.5.1. Excepciones

Las excepciones se utilizan para informar errores de acceso a datos y datos inválidos sin confundirlos con un resultado exitoso. En la capa DAO, las excepciones `SQLException` se capturan y se encapsulan en `ErrorAccesoDatosException`. En la capa de Control, los Gestores validan los datos de entrada y utilizan `ValidacionException` para casos como un DNI inválido, datos obligatorios ausentes o un cliente inexistente. De esta forma, cada capa conserva su responsabilidad: el DAO comunica problemas de infraestructura y el Gestor comunica problemas de negocio u operación.

3.5.2. Uso de `ArrayList`

La clase `ArrayList` se utiliza cuando la cantidad de elementos es variable y se necesita conservar una colección ordenada para procesarla o presentarla. Por ejemplo, `ClienteDAO.listarTodos` agrega a un `ArrayList` los clientes obtenidos del `ResultSet`; de forma similar, los gestores y las vistas trabajan con listas de compras, beneficios y filas de reportes. Esta estructura permite recorrer los resultados con ciclos, agregar elementos durante la consulta y entregar colecciones a la interfaz sin fijar de antemano una cantidad máxima de registros.

3.5.3. Manejo de Archivos

No se utilizó persistencia en archivos de texto porque la naturaleza del sistema es transaccional y puede requerir operaciones concurrentes sobre clientes, compras y beneficios. Para este escenario se necesita un motor de base de datos relacional como MySQL, capaz de mantener la integridad referencial y las propiedades ACID (atomicidad, consistencia, aislamiento y durabilidad). Además, el alcance de este MVP no incluye requerimientos de exportación o importación externa; por lo tanto, incorporar archivos de texto habría duplicado mecanismos de persistencia sin aportar una funcionalidad necesaria para la versión presentada.